

Overview

Part 10 — AI Architecture · 10-ai-architecture/README.md

Module 10 — AI Architecture

Where you are: Stop 10 of 16 on the end-to-end AI track. **Time:** ~20–24 hours · **Prereq:** Modules 04 (LLM [04-llm]), 06 (Agents [06-ai-agents]), 07 (Agentic AI [07-agentic-ai]), 08 (RAG [08-rag]) and 09 (MCP [09-mcp]) — this module assumes you already know what each component *does* and teaches you how to wire them into a system that survives production.

Every module before this one taught you a component: a model, an agent loop, a retriever, a protocol for tools. None of them, alone, is a product. This module is where you stop asking "how does RAG work" and start asking "where does the vector DB call live relative to the rate limiter, and what happens when the embedding provider is down at 2 a.m." That question — and the fifteen like it — is systems integration, and it is a different skill from model literacy. Get the architecture wrong and a well-evaluated model still ships an unreliable, unauditable, unboundedly expensive product.

Learning objectives

By the end of this module you can:

1. Draw the layered reference architecture for an LLM-powered product from client to observability, and name what leaks across each layer boundary when the system is built carelessly.
2. Design a model gateway that does cost-tiered routing, provider failover, and circuit breaking — and defend build-vs-buy against a LiteLLM-style gateway or a hosted router.
3. Choose the right cache at the right layer (exact-match, semantic, prefix, embedding) and explain, with numbers, why a semantic cache is a correctness trade-off, not a free win.
4. Design the RAG ingestion pipeline for freshness, and select a vector database against concrete criteria instead of vibes or a blog post ranking.
5. Apply reliability patterns — idempotency, circuit breakers, graceful degradation, multi-provider redundancy — to long-running agent workloads, and explain the re-execution trap.
6. Build one observability architecture (traces, logs, metrics, cost) that ships *before* launch, and place security controls precisely in the request path rather than bolting them on afterward.
7. Draw and defend one complete, labeled, production-grade reference architecture for a named system.

Suggested path

#	Do this	File	Time
1	The layered architecture, routing, caching, data	notes/01-reference-architecture.md	4h
2	Reliability, observability, security, deployment, cost, capstone	notes/02-production-patterns.md	5h
3	Build and break a model gateway	code/model_gateway_simulator.py	2h
4	Break a semantic cache on purpose	code/semantic_cache_correctness.py	2h
5	Slides	slides/	1h
6	Lab	lab/EXERCISES.md	6h
7	Papers & references	papers/PAPERS.md	4h

The 18 terms you must own

BFF (backend-for-frontend) · model gateway · cost-tiered routing · circuit breaker · semantic cache · prefix/prompt cache · feature store · hybrid search · re-index freshness · idempotency key · graceful degradation · shadow deployment · canary deployment · prompt registry · blast radius · backpressure · SSE (server-sent events) · defense in depth

Exit check

You can hand a colleague one labeled architecture diagram for a named system (e.g. "internal enterprise RAG + agent assistant with tool access") that shows every layer, every cache, the routing and failover path, where security controls sit, where observability hooks in, and how a prompt or model change gets deployed and rolled back — plus a one-paragraph rationale for each box that a reviewer could not poke a hole in during a 15-minute design review.